

Lecture 14: Measuring performance (cont'd)

CMPE I20: Computer Organization & Architecture

Olivia Weng

Logistics

- Reminder: check-in with professor is 1% of your final grade
 - Sign up [link](#)
- Midterm Exam in-class next Tuesday, October 13th
 - Can bring 1 double-sided 8.5x11" note sheet (handwritten notes highly recommended!)
 - Cannot bring any other aids (e.g., no calculators, extra notes, etc)
- HW3 is due October 15th, 11:59pm

Amdahl's corollaries

- Make the common case fast (i.e., x should be large!)
 - Common == “most time consuming” not necessarily “most frequent”
 - The uncommon case is usually not as important.
 - Be sure of what the common case is. Measure it!
 - Your intuition is probably wrong.
 - The common case can change based on inputs, compiler options, etc.
- Repeat...
 - With optimization, the common becomes less common.
 - An uncommon case will become the new common case.
 - Now you have a new target for optimization.

What are some metrics we can measure?

- Power
 - Performance
 - Latency — time
 - Throughput — # stuff / s
 - Area — how much memory?
 - Cost — storage?
- registers?
- } resource consumption.

$$1 \text{ Hz} = \frac{1}{1\text{s}} \quad 1 \text{ GHz} = \frac{10^9}{1\text{s}} = \frac{1}{10^{-9}} \text{ s}$$

Measure latency: the CPU Performance Equation (PE)

- Execution time (ET) = IC x CPI x CT

- IC: instruction count $\rightarrow$ # of instr. executed
- CPI: cycles per instruction $\rightarrow$ # cycles per 1 instr.
- CT: cycle time $\rightarrow$ time it takes to execute 1 cycle
 - $\hookrightarrow 1 \text{ GHz} \Rightarrow CT = 1 \text{ ns} = \frac{1}{10^9} \text{ s}$
 - $2 \text{ GHz} \Rightarrow CT = 0.5 \text{ ns} = \frac{1}{2 \times 10^9} \text{ s}$

Performance equation (PE) practice problem

- What is the execution time (ET?)
 - A loop consists of 10 instructions.
 - It executes 10^8 times.
 - Each loop iteration takes 12 cycles (on average).
 - 2 GHz clock

- A. 0.6 s ✓
 - B. 2.4×10^{18} s
 - C. 2.4 s
 - D. 4.2 s
 - E. 6.0 s

$$ET = \text{Instr. count} \times \text{Cycles per instr} \times \text{Cycle time}$$

$$IC = 10 \times 10^8 = 10^9$$

$$CPI = \frac{12 \text{ cycles}}{10 \text{ instr}} = \frac{6}{5} \text{ CPI}$$

$$CT = \frac{1}{2 \times 10^9} \text{ s}$$

$$IC \times CPI \times CT$$

$$\Rightarrow 10^9 \times \frac{6}{5} \times \frac{1}{2 \times 10^9}$$

$$\Rightarrow \frac{6}{10} = \boxed{0.6 \text{ s}}$$

Implications of the Performance Equation (PE)

- Intuition: ET increases/decreases linearly with
 - IC ✓
 - CPI ✓
 - CT ✓
- We can improve performance by improving each term
 - Benefits will compound!

Who can impact Instruction Count (IC)?

A	ISA Designer Compiler Programmer
B	Hardware designer
C	Hardware Designer ISA Designer
D	Programmer Hardware Designer
E	ISA Designers Compiler Hardware Designers

Who can impact Instruction Count (IC)?

A	ISA Designer Compiler Programmer
B	Hardware designer
C	Hardware Designer ISA Designer
D	Programmer Hardware Designer
E	ISA Designers Compiler Hardware Designers

RISC vs CISC

- RISC: Reduced Instruction Set Computer
 - RISC-V is a RISC architecture (in the name!)
 - Fixed instruction length & few instruction formats
 - Load/store architecture
 - Enable pipelining & parallelism!
- CISC: Complex Instruction Set Computer
 - Provide more powerful operations
 - Reduce instructions executed
 - Danger: slower cycle time / higher CPI
 - Ex: x86

RISC vs CISC

- Virtually all new instruction sets since 1982 have been RISC
- Why does x86 still dominate?
 - Historical dominance
 - Complexity:
 - Instructions 1-17 bytes long
 - 1 operand acts as source and destination
 - 1 operand can come from memory
 - But, in fact, x86's CISC instructions actually get decoded into micro-operations (uops)
 - RISC-like!
 - Compilers avoid portions of ISA that are slow

Dynamic vs static instruction count

- The instruction count in the PE is the dynamic instruction count
- Dynamic instruction count
 - The number of instructions that actually ran during program execution
 - Ex: When I ran Program A, it executed 1 million instructions
- Static instruction count
 - Fixed at compile time, referring to the program as it was compiled
 - Ex: This function compiled to 10 instructions.

Who can impact Cycles per Instruction (CPI)?

A	ISA Designer Compiler Programmer
B	Hardware designer
C	Hardware Designer ISA Designer
D	Programmer Hardware Designer
E	ISA Designers Compiler Hardware Designers Programmer

Who can impact Cycles per Instruction (CPI)?

A	ISA Designer Compiler Programmer
B	Hardware designer
C	Hardware Designer ISA Designer
D	Programmer Hardware Designer
E	ISA Designers Compiler Hardware Designers Programmer

Improving CPI

- CPI is the **most complex term** in the PE because **many things impact** it:
 - Compiler
 - Program inputs
 - Processor design
 - Memory system
 - Processor's ISA
- **Different instructions** require a **different amount of work**
 - Instructions that do more work generally increase CPI

Instruction mix affects CPI

- Program

- Floating point instructions have larger CPI
- Memory-intensive instructions have larger CPI

- Program inputs

- Different inputs will exercise different parts of the program

- Compiler

- Compiler can choose which instruction to use, e.g., shift left vs multiply

⇒ cheaper!

Who can impact Cycle Time (CT)?

A	ISA Designer Compiler Programmer
B	Hardware designer
C	Hardware Designer ISA Designer
D	Programmer Hardware Designer
E	ISA Designers Compiler Hardware Designers Programmer

Who can impact Cycle Time (CT)?

A	ISA Designer Compiler Programmer
B	Hardware designer
C	Hardware Designer ISA Designer
D	Programmer Hardware Designer
E	ISA Designers Compiler Hardware Designers Programmer

What impacts CT?

- Processor design
 - Processor architecture and microarchitecture determine minimal cycle time